



Actividad:
Proyecto Final

Profesor:
Colchado Ruiz, Gerardo

INTEGRANTES	CÓDIGO
Cayllahua Hilario, Joel Modesto	202410731
Guerrero Gutierrez, Nayeli Belén	202410790
Maquera Quispe, Luis Fernando	202410621
Sánchez Terrazos, Leonardo Gabriel	202410273
Tamayo Hilario, Maria Karolay	202410766

Diagrama de arquitectura:

La solución implementada sigue una arquitectura **Event-Driven** (basada en eventos) y Serverless, utilizando los siguientes componentes clave:

- **Frontend:** Aplicación Web Single Page Application (SPA) desarrollada en **React** con **TypeScript** y **TailwindCSS**, desplegada mediante **AWS Amplify**.
- **Backend:** 7 Microservicios independientes contruidos sobre **AWS Lambda** y **Node.js 20.x**.
- **Base de Datos:** Amazon **DynamoDB** con diseño de tabla única (*Single Table Design*) o tablas por servicio según necesidad.
- **Comunicación:**
 - **Síncrona:** Amazon API Gateway (HTTP APIs).
 - **Asíncrona:** Amazon EventBridge (Bus de eventos), Amazon SQS (Colas de mensajes) y Amazon SNS (Notificaciones).
 - **Tiempo Real:** Amazon API Gateway (WebSocket APIs) para actualizaciones en vivo de estados de órdenes.
- **Orquestación:** **AWS Step Functions** para flujos de negocio complejos como el procesamiento de órdenes.

Diagrama de Arquitectura (Conceptual): La arquitectura sigue el flujo: Cliente -> API Gateway -> Lambda -> DynamoDB -> EventBridge -> WebSocket -> Cliente/Cocina/Delivery3. Detalles del Backend (Microservicios)

El sistema está dividido en 7 microservicios principales, gestionados mediante el **Serverless Framework**: 3.1. E-commerce Service

Encargado de la experiencia del cliente final.

- **Funciones:** Registro/Login (Auth), Gestión de Carrito, Catálogo de Menú, Creación de Órdenes.
- **Tecnologías:** Lambda, DynamoDB, JWT para autenticación.

3.2. Kitchen Service

Gestiona el flujo de preparación de pedidos en cocina.

- **Funciones:** Visualización de comandas, asignación de chefs, actualización de estados (En preparación, Listo).
- **Tecnologías:** Lambda, DynamoDB, WebSocket (para recibir alertas de nuevas órdenes).

3.3. Delivery Service

Administra la logística de entrega.

- **Funciones:** Gestión de repartidores (*Drivers*), asignación de pedidos, *tracking* de entrega.
- **Tecnologías:** Lambda, DynamoDB, Geo-localización (simulada).

3.4. Admin Service

Panel administrativo para la gestión del negocio.

- **Funciones:** *Dashboard* de métricas, gestión de sedes (*tenants*), usuarios y productos.
- **Tecnologías:** Lambda, DynamoDB.

3.5. WebSocket Service

Servicio dedicado a la comunicación en tiempo real.

- **Funciones:** Gestión de conexiones, *broadcast* de eventos (ej. "Tu pedido está listo").
- **Integración:** Conectado a EventBridge para reaccionar a cambios de estado en cualquier parte del sistema.

3.6. Step Functions Service

Orquestador de flujos de negocio críticos.

- **Workflow: OrderWorkflow**
 1. Validar stock y carrito.
 2. Procesar pago (simulado).
 3. Persistir orden.
 4. Publicar evento *OrderCreated*.

3.7. Workers Service

Procesamiento en segundo plano.

- **Funciones:** Consumidores de colas SQS para tareas pesadas o desacopladas (ej. generación de reportes, notificaciones masivas).

4. Detalles del Frontend

La interfaz de usuario se ha desarrollado priorizando la experiencia de usuario (UX) y el rendimiento.

- **Stack Tecnológico:**
 - **Framework:** React 18 (Vite).
 - **Lenguaje:** TypeScript para tipado estático y robustez.
 - **Estilos:** TailwindCSS para diseño responsivo y moderno.
 - **Estado Global:** React Context API / Hooks.
- **Módulos:**
 - **Cliente:** Catálogo, Carrito, Checkout, *Tracking* de pedido.
 - **Dashboard Cocina:** Vista Kanban de órdenes en tiempo real.
 - **Dashboard Delivery:** Mapa y lista de pedidos para repartidores.
 - **Dashboard Admin:** Gráficos y tablas de gestión.

5. Conclusiones

1. **Escalabilidad:** La arquitectura Serverless permite escalar a cero (costo cero cuando no hay uso) y escalar automáticamente ante picos de demanda sin intervención manual.

2. **Desacoplamiento:** El uso de EventBridge y SQS permite que los servicios (Cocina, Delivery) evolucionen independientemente sin afectar al E-commerce.
3. **Tiempo Real:** La implementación de WebSockets mejora significativamente la experiencia del usuario final y la eficiencia operativa en cocina.
4. **Seguridad:** Se implementaron roles (RBAC) y validaciones estrictas de `tenant_id` para asegurar el aislamiento de datos entre sedes.

